Mukesh Patel School of Technology Management & Engineering, Mumbai

Department of Computer Science and Engineering (Data Science) | Academic Year 2024–25

Architectural Defect Classification and Severity Assessment via Hybrid CNN–Classical Feature Fusion

Saanvi Kanodia (L014), Nidhi Shah (L025), Mysha Bhartiya (L033), Muskaan Saluja (L035)

CSEDS – 311

Abstract. Structural defects in buildings—dampness, mold growth, surface cracks, spalling, and paint peeling—are a major challenge for building maintenance and safety. Traditional inspection methods rely on manual visual assessment, which is time-consuming, inconsistent, and heavily dependent on the experience of the inspector. This is especially problematic when distinguishing between defect types that look visually similar, such as spalling and peeling.

This project presents an automated system to classify wall defects and estimate their severity using a combination of deep learning and classical image processing. The system operates on still images sourced from a labelled dataset of building wall photographs, rather than a live capture pipeline. The deep learning component uses a MobileNetV2 model pre-trained on ImageNet and fine-tuned on this dataset. The classical component extracts 16 hand-crafted features based on texture, color, frequency, and edge properties. These two components are fused to produce the final prediction. The dataset comprises 400 images per defect class across five categories (damp, mold, crack, spalling, peeling), partitioned into 70% training, 15% validation, and 15% test splits. On the balanced test set of 298 images, the hybrid model achieves **89% accuracy** and a **macro F1-score of 0.89**, compared to 79% for an XGBoost baseline using classical features alone. The system also estimates defect severity and outputs a Wall Health Index (0–100), with Grad-CAM heatmaps providing visual explanations of model decisions.

Keywords: Wall defect detection, MobileNetV2, feature fusion, transfer learning, severity assessment, Grad-CAM, image processing.

---

1 Introduction

1.1 Background and Motivation

Buildings deteriorate over time, and surface defects such as dampness, mold, cracks, spalling (where chunks of material break away from the surface), and paint peeling are among the most visible and common signs of this deterioration. If these defects are not detected and repaired early, they can grow from minor cosmetic issues into serious structural problems. Dampness and mold also create health risks for building occupants—prolonged exposure is associated with respiratory issues and other health complications [1]. Undetected cracks, on the other hand, can signal deeper foundational damage that puts the structural integrity of a building at risk.

The economic cost of building maintenance and repair is substantial. Studies in building science estimate that the cost of corrective repair is several times higher than the cost of preventive maintenance [2]. Early and accurate detection of defects is therefore not only a safety concern but also a significant financial one.

Currently, buildings are inspected manually by trained engineers or surveyors. While these inspections are essential, they are slow, expensive, and the results can vary depending on who is conducting the inspection. Two different inspectors examining the same wall may record different findings. This inconsistency is a well-documented problem in the field [3], and it limits the reliability of inspection data for maintenance planning.

1.2 Problem Statement

The goal of this project is to develop an automated system that can process a wall image drawn from a structured dataset and:

1. Correctly identify which type of defect is present, from five categories: damp, mold, crack, spalling, and peeling.
2. Estimate how severe the defect is.
3. Provide a visual explanation of which part of the image led to the classification.

The most challenging part of this problem is distinguishing between spalling and peeling. Both involve surface material separating from the wall, and in photographs they can appear very similar. A reliable classifier must capture subtler cues—such as texture roughness, color saturation, and edge character—to separate them correctly.

1.3 Proposed Approach and Contributions

We propose a hybrid dual-branch fusion model that combines two types of analysis. A **deep learning branch** uses MobileNetV2 to learn spatial and contextual patterns

directly from images. A **classical feature branch** extracts 16 carefully designed numerical descriptors tied to the physical properties of each defect type. The outputs of both branches are merged before the final prediction. This hybrid approach outperforms either method used alone.

The main contributions of this project are:

- A dual-branch fusion model achieving **89% test accuracy** on a five-class wall defect dataset.
- A two-phase training strategy that fine-tunes MobileNetV2 without corrupting its pre-trained features.
- A preprocessing pipeline using CLAHE and bilateral filtering to enhance defect visibility while preserving color information.
- A severity assessment module producing class-specific severity scores and a **Wall Health Index** (0–100).
- A comparison against an XGBoost baseline showing a **10 percentage point improvement** from the deep learning branch.
- Grad-CAM visualizations making model predictions interpretable and verifiable.

2 Related Work

2.1 Traditional Methods for Defect Detection

Before deep learning, automated crack and defect detection relied on hand-crafted image processing. Canny edge detection [4] and Sobel filters were used to find crack-like linear patterns. Morphological operations such as dilation and erosion were applied to highlight or suppress structural features. Otsu thresholding [5] provided an automatic way to separate defect regions from the background based on gray-level histograms.

While computationally simple, these methods have a critical weakness: they must be manually tuned for each new environment, and they degrade significantly when lighting conditions change or surfaces vary in texture.

Texture-based methods offered some improvement. GLCM (Gray-Level Co-occurrence Matrix) features [6] capture spatial relationships between pixel intensities, useful for detecting surface roughness and damage. LBP (Local Binary Patterns) [7] are rotation-invariant texture descriptors that can distinguish smooth damp surfaces from rough spalling. DCT (Discrete Cosine Transform) energy features [8] separate smooth surfaces from rough or damaged ones based on the distribution of high-frequency energy.

2.2 Deep Learning for Defect Detection

Convolutional neural networks (CNNs) transformed automated defect detection. Cha et al. [9] demonstrated that CNNs could automatically learn which visual patterns are associated with cracks, without requiring any manually engineered features. This approach significantly outperformed classical methods and established CNNs as the standard for visual inspection tasks.

Transfer learning [10] further improved results in settings where domain-specific training data is limited. Rather than training a CNN from scratch, models pre-trained on large datasets such as ImageNet are adapted to the target task. The low-level features learned on natural images—edges, textures, gradients—transfer well to building defect images. MobileNetV2 [11] is a particularly attractive choice for this application: it is specifically designed to be lightweight and efficient, making it suitable for deployment on mobile devices or in environments with limited computing resources, while still delivering high accuracy.

Grad-CAM [12] was introduced to address the interpretability gap in CNN predictions. It computes a heatmap showing which image regions most influenced the model's output. This is particularly valuable in engineering applications where human inspectors need to verify and understand model decisions before acting on them.

2.3 Multi-Class Defect Classification

Most prior work has focused on detecting a single defect type, typically cracks [14]. Classifying multiple overlapping defect types simultaneously has received considerably less attention. Dung and Anh [15] worked on pixel-level crack segmentation using fully convolutional networks. Liu et al. [16] classified multiple pavement defect types using multiscale CNNs. However, neither study specifically addressed the spalling–peeling ambiguity, and neither included a severity estimation component.

2.4 Hybrid Approaches

Some researchers have combined CNN-derived features with classical hand-crafted descriptors [17]. This approach leverages the strengths of both: the CNN provides rich global context about the image, while classical features provide interpretable, physically grounded information. Gradient boosting classifiers such as XGBoost [18] have been used with hand-crafted features as competitive baselines [19]. Our work builds on this hybrid philosophy and demonstrates its value specifically for the wall defect classification problem.

3 Classical Feature Vector

In addition to the deep learning model, 16 numerical features are extracted from each image. These are designed to capture the specific visual properties associated with each defect class, as summarized in Table 1.

Table 1: 16 Classical Features and Their Role in Defect Classification

#	Feature	What it captures
1	DCT Energy	High for rough defects (spalling, peeling); low for smooth surfaces (damp, mold)
2	Edge Density	High for cracks (thin, linear edges); distinguishes cracks from other defects
3	Saturation Mean	High for mold (greenish/brownish colour); low for damp and crack surfaces
4	Local Variance	High for rough spalling; low for smoother surfaces like peeling or damp
5	Intensity Mean	Low for damp walls (large dark wet patches)
6	Green Ratio	>1 for mold; neutral for all other classes
7	Dark Ratio	Fraction of very dark pixels; high for damp regions
8	Hue Std. Dev.	Low for mold (narrow hue range); high for damp (wide colour variation)
9	LBP Entropy	High for complex textures (mold colonies, rough spalling)
10	GLCM Contrast	Surface roughness; high for spalling
11	GLCM Correlation	Structural regularity; high for uniform/ordered surfaces
12	GLCM Energy	Uniformity; high for smooth damp surfaces
13	GLCM Homogeneity	High for smooth surfaces; low for crack-disrupted surfaces
14	Sobel Mean	Overall edge strength; high for peeling and crack surfaces
15	LAB a^* Mean	Green-red colour axis; positive values indicate mold
16	LAB b^* Mean	Blue-yellow colour axis; captures stain tones from mold and damp

3.1 DCT Energy

The DCT energy ratio measures how much of the image's energy is concentrated in high-frequency components, which correspond to sharp edges and rough textures. It is computed over non-overlapping 32×32 blocks of the grayscale image:

$$e_{\text{DCT}} = \frac{\sum_{b} \sum_{\substack{(i,j)\,:\,i \geq N/2 \\ \text{or } j \geq N/2}} D_b(i,j)^2}{\sum_{b} \sum_{i,j} D_b(i,j)^2 + \epsilon} \tag{1}$$

where D_b is the DCT of block b, $N = 32$, and $\epsilon = 10^{-8}$ prevents division by zero. Rough surfaces such as spalling and peeling yield high values; smooth surfaces like damp and mold yield low values.

3.2 LBP Entropy

Local Binary Pattern (LBP) entropy quantifies the complexity of local texture. Higher entropy indicates a more varied and spatially complex texture:

$$h_{\text{LBP}} = -\sum_{k=0}^{P+1} p_k \log_2(p_k + \epsilon) \tag{2}$$

where p_k is the normalized frequency of LBP pattern k, computed with radius $R = 1$ and $P = 8$ sample points.

3.3 GLCM Features

GLCM (Gray-Level Co-occurrence Matrix) features capture how pixel intensities are spatially related across the image. Four statistics are extracted—contrast, correlation, energy, and homogeneity—averaged over multiple angles and distances on an 8-level quantized grayscale image. These features are particularly effective for distinguishing surface roughness (spalling) from structural regularity (crack) and surface uniformity (damp).

3.4 Validation

To verify that these features carry useful discriminative information before any deep learning is involved, the classical branch was tested alone on representative images from each class. In all cases, the correct class received the highest probability score (approximately 0.5 or above), confirming that the 16 features provide a meaningful signal for classification.

4 Preprocessing Pipeline

A four-step preprocessing pipeline is applied to each image before it is fed into the model. The goal is to improve defect visibility while preserving color information, which is critical for identifying mold and dampness.

4.1 Step 1: Convert RGB to HSV

The image is first converted from RGB to HSV (Hue, Saturation, Value). This separates brightness (the Value channel) from color (Hue and Saturation), allowing contrast enhancement to be applied to brightness alone without altering the color information that distinguishes mold (greenish hues) from damp (dark, desaturated tones).

4.2 Step 2: Apply CLAHE to the Value Channel

CLAHE (Contrast Limited Adaptive Histogram Equalization) [13] is applied to the Value channel with a clip limit of 2.0 and an 8×8 tile grid. Unlike global histogram equalization, CLAHE operates on small local regions and caps the amplification of noise. This improves contrast in dark damp patches and low-contrast crack regions without over-brightening areas that are already well-exposed.

4.3 Step 3: Convert Back to RGB and Apply Bilateral Filter

The enhanced image is converted back to RGB. A bilateral filter ($d = 9$, $\sigma_{\text{color}} = 75$, $\sigma_{\text{space}} = 75$) is then applied. The bilateral filter smooths noise in uniform regions while preserving sharp edges. This is important because blurring defect boundaries would degrade the edge-based features (Edge Density, Sobel Mean) used in classification.

4.4 Step 4: Resize and Normalise

All images are resized to 256×256 pixels and pixel values are scaled to $[0, 1]$ by dividing by 255. This ensures a consistent input size across the dataset and promotes stable gradient behavior during training.

5 Model Architecture

5.1 Overall Design

The model uses a dual-branch fusion architecture. Two separate branches process different representations of the same image and their outputs are merged before the final classification. This design allows the model to simultaneously exploit the spatial reasoning of a CNN and the physically grounded information in the classical features.

5.2 CNN Branch

The CNN branch takes the preprocessed $256 \times 256 \times 3$ image as input and uses **MobileNetV2** [11] as the backbone. MobileNetV2 was pre-trained on ImageNet and is designed to be lightweight and efficient. Its inverted residual structure with linear bottlenecks allows it to extract rich hierarchical features while keeping the model compact.

The backbone outputs an $8 \times 8 \times 1280$ feature map, which is compressed to a 1280-dimensional vector by Global Average Pooling. A Dropout layer (rate = 0.30) is applied to reduce overfitting.

5.3 Classical Feature Branch

The classical branch takes the 16-dimensional feature vector as input and transforms it through two fully connected layers:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{v} + \mathbf{b}_1) \in \mathbb{R}^{64}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\mathbf{h}_1' + \mathbf{b}_2) \in \mathbb{R}^{32}$$

Batch Normalization is applied between the two layers to stabilize training and reduce sensitivity to the scale differences between features (e.g., DCT energy values versus integer pixel counts).

5.4 Fusion and Output

The 1280-dimensional CNN output and the 32-dimensional classical branch output are concatenated into a single 1312-dimensional vector:

$$\mathbf{f} = [\mathbf{g}_{\text{CNN}};\, \mathbf{h}_2] \in \mathbb{R}^{1312}$$

This combined vector is passed through a Dense layer of 128 neurons with 50% Dropout, followed by a final output layer with 5 neurons and Softmax activation, producing a probability for each of the five defect classes.

5.5 Grad-CAM Visualizations

To explain model predictions, Grad-CAM [12] is applied to the final convolutional layer of MobileNetV2. It computes the gradient of the class score with respect to each feature map and uses these gradients to weight the feature maps:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k A^k\right)$$

where α_k is the importance weight for feature map A^k, computed by global average pooling the gradients. The resulting heatmap is overlaid on the original image, highlighting which regions of the wall the model focused on when making its prediction.

6 Training

6.1 Dataset

The dataset consists of 400 images per class across five defect categories—`damp`, `mold`, `crack`, `spalling`, and `peeling`—giving 2,000 images in total. All images are still photographs sourced from a pre-existing labelled repository of building wall defects; no live capture pipeline was used. Each image shows a single defect type in isolation, with the class label determined by the source folder.

The dataset is partitioned per class into **70% training** (280 images), **15% validation** (60 images), and **15% test** (60 images), yielding 1,400 training images, 300 validation images, and 300 test images overall. The test set (298 usable images after preprocessing checks) is held out entirely during training and used only for final evaluation, providing a balanced assessment across all five classes.

6.2 Data Augmentation

Because building defect datasets are typically small, data augmentation is applied during training to artificially increase the variety of training images:

- Random horizontal and vertical flips
- Brightness adjustment ($\pm 25\%$)
- Contrast adjustment (scale factor between 0.7 and 1.3)
- Saturation and hue jitter
- Random crop and resize (retaining 90–100% of the original image)

Augmentation is applied only during training. The 16 classical features are extracted from the preprocessed image before augmentation, so they remain stable and are not distorted by the random transforms.

6.3 Phase 1: Training the Fusion Head

In the first training phase, the MobileNetV2 backbone is frozen and only the classical branch and fusion head layers are updated. The Adam optimizer is used with a learning rate of 10^{-3} for up to 12 epochs.

This approach protects the pre-trained backbone weights during the early stages of training. If the backbone were unfrozen immediately, the large gradients from a randomly initialized head could corrupt the low-level features MobileNetV2 learned from ImageNet. Training the head first ensures that, when the backbone is eventually unfrozen in Phase 2, the incoming gradients are already well-structured and small.

6.4 Phase 2: Fine-Tuning the Backbone

In the second phase, the top 30 layers of MobileNetV2 are unfrozen and trained together with the rest of the model. A much lower learning rate of 10^{-5} is used to make small, careful adjustments to the backbone. Training runs for up to 18 additional epochs.

6.5 Training Callbacks

Three callbacks are used to improve training efficiency and prevent overfitting:

- **Early Stopping:** Halts training if validation accuracy does not improve for 6 consecutive epochs, and restores the best weights seen during training.
- **Reduce LR on Plateau:** Halves the learning rate if validation accuracy stalls for 3 consecutive epochs.
- **Model Checkpoint:** Saves the model weights whenever a new best validation accuracy is achieved.

6.6 XGBoost Baseline

An XGBoost classifier [18] was trained using only the 16 classical features, with no image input. This baseline isolates the contribution of the deep learning branch and provides a concrete quantification of how much the CNN adds to the system.

7 Severity Assessment

Once a defect is classified, the system estimates how severe it is. Each defect class uses a class-specific formula that reflects the geometric and visual properties most relevant to that type of damage. The result is a severity score $S \in [0, 100]$.

7.1 Crack Severity

Cracks are detected using a combination of black-hat morphological transforms and multi-scale Canny edge detection. The crack skeleton is extracted using image thinning [20] to measure crack length in pixels. A crack density relative to image diagonal is then computed:

$$\rho_{\text{crack}} = \frac{L_{\text{px}}}{\sqrt{H^2 + W^2}}, \qquad S_{\text{crack}} = \min(100,\ A_{\%} \times 2 + \rho_{\text{crack}} \times 200)$$

This formulation ensures that a long, thin crack—structurally more serious than a short wide crack of equal area—receives a proportionally higher severity score.

7.2 Mold Severity

Mold regions are identified by a color filter in HSV space targeting green-brown hues, followed by morphological cleanup. The number of distinct mold colonies N_c is counted as a proxy for spread:

$$S_{\text{mold}} = \min(100,\ A_{\%} \times 1.5 + N_c \times 2)$$

7.3 Damp Severity

Dark wet patches are segmented using multi-level Otsu thresholding [5]. A texture filter is applied to prevent false positives on structured surfaces such as brickwork:

$$S_{\text{damp}} = \min(100,\ A_{\%} \times 1.5)$$

7.4 Spalling Severity

Spalling regions are identified by locating high-variance areas with low color saturation, since exposed concrete or masonry is typically desaturated:

$$S_{\text{spalling}} = \min(100,\ A_{\%} + e_{\text{DCT}} \times 50)$$

7.5 Peeling Severity

Peeling regions are identified by combining high local variance, strong horizontal edges, and relatively higher color saturation (lifted paint retains color, unlike exposed concrete):

$$S_{\text{peeling}} = \min(100,\ A_{\%} \times 1.2 + e_{\text{DCT}} \times 40)$$

7.6 Wall Health Index

All severity scores are converted into a single **Wall Health Index**:

$$H = 100 - S, \qquad H \in [0, 100]$$

The four health categories are:

- $H \geq 75$: **GOOD** — No immediate action required
- $50 \leq H < 75$: **MODERATE** — Monitor and plan maintenance
- $25 \leq H < 50$: **POOR** — Maintenance required soon
- $H < 25$: **CRITICAL** — Immediate repair needed

8 Results

8.1 Hybrid Model Performance

The hybrid CNN–classical model was evaluated on a balanced test set of 298 images. Table 2 shows per-class precision, recall, and F1-score, and Figure 1 shows the corresponding confusion matrix.

Table 2: Hybrid Model — Per-Class Test Set Results

Class	Precision	Recall	F1-Score	Images
Damp	0.90	1.00	0.95	57
Mold	0.92	0.90	0.91	61
Crack	0.93	0.90	0.92	60
Spalling	0.88	0.73	0.80	60
Peeling	0.84	0.93	0.88	60
Overall Accuracy		**89%**		298
Macro Average	0.89	0.89	0.89	298

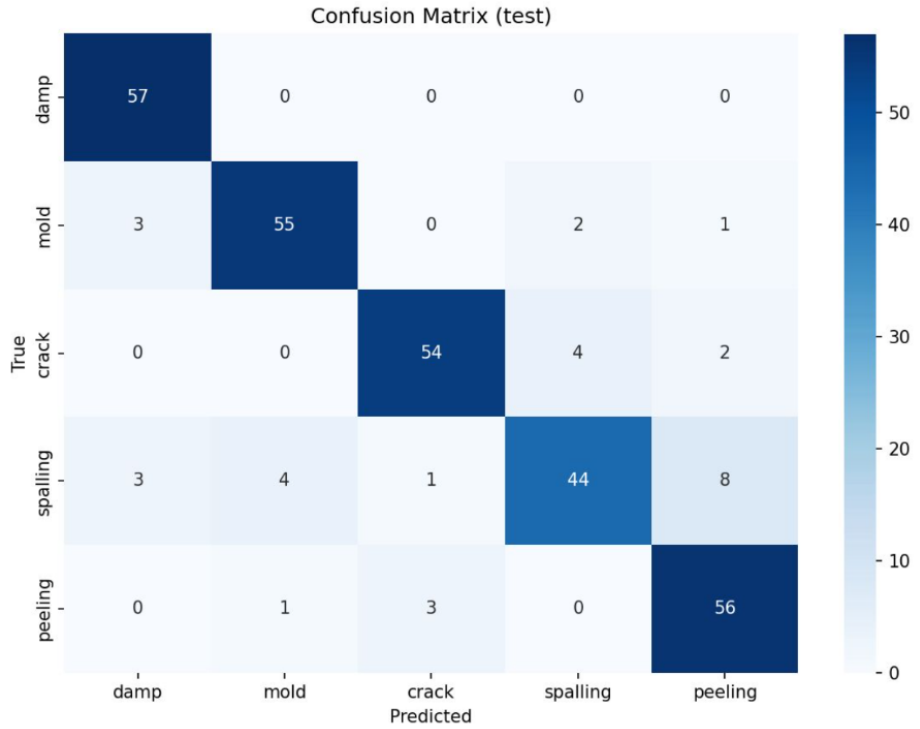


Figure 1: Confusion matrix on the 298-image test set. Rows denote true labels; columns denote predicted labels. The diagonal entries show correct classifications. Spalling is the most challenging class, with 8 instances misclassified as peeling due to their visual similarity.

Key observations from Table 2 and Figure 1:

- **Damp** achieved perfect recall (57 out of 57 correctly identified), confirming that intensity and GLCM energy features are highly effective for this class.
- **Crack** achieved the highest precision (0.93). Its few misclassifications were spread across spalling and peeling, which share similar edge patterns.
- **Spalling** was the hardest class (F1 = 0.80), with 8 instances misclassified as peeling. This is expected given the visual similarity between the two classes.

8.2 XGBoost Baseline Results

Table 3 shows the results of the XGBoost model trained on only the 16 classical features.

Table 3: XGBoost Baseline — Per-Class Validation Results

Class	Precision	Recall	F1-Score	Images
Crack	0.77	0.83	0.80	60
Damp	0.98	0.98	0.98	55
Mold	0.81	0.73	0.77	60
Peeling	0.69	0.78	0.73	60
Spalling	0.70	0.62	0.65	60
Overall Accuracy		**79%**		295
Macro Average	0.79	0.79	0.79	295

8.3 Model Comparison

Table 4 summarizes the comparison between the two models.

Table 4: Performance Comparison: Hybrid Model vs. XGBoost

Model	Accuracy	Macro F1	Worst Class F1
XGBoost (classical only)	79%	0.79	0.65 (spalling)
Hybrid CNN–Classical	**89%**	**0.89**	**0.80** (spalling)
Improvement	+10%	+0.10	+0.15

9 Discussion

9.1 Why the Hybrid Approach Works

The XGBoost experiment clearly illustrates the complementary roles of the two branches. Classical features alone are highly effective for defects with strong, distinctive visual signatures. Damp, for example, achieves an F1-score of 0.98 with XGBoost alone, because

dark wet patches produce very consistent intensity, GLCM energy, and dark ratio values. Similarly, crack achieves F1 = 0.80 from edge density and Sobel features.

The limitation of classical features becomes apparent with spalling and peeling. Both defects produce rough surfaces with high DCT energy and high local variance, making them nearly indistinguishable based on these statistics alone. The CNN branch addresses this by capturing spatial context—the arrangement of edges, the distribution of texture patches, the geometry of lifted regions—that cannot be expressed as a single scalar feature. This spatial understanding is what drives the 15-point F1 improvement on spalling.

9.2 Interpretability

Grad-CAM heatmaps confirmed that the model learned to focus on the correct regions of each image. For crack images, attention was concentrated along the crack line. For damp, the model highlighted the dark wet regions. For mold, activation was strongest over greenish-brown patches. For peeling, the model focused on the edges where paint was visibly lifting from the surface. This consistency between the model's attention and the physically relevant regions makes predictions trustworthy and easy for a human inspector to verify.

9.3 Severity Assessment

The severity module extends the system beyond simple classification to provide actionable maintenance information. The crack severity formula is particularly noteworthy: by incorporating skeleton length relative to image diagonal, it assigns a higher score to long thin cracks than to short wide ones of equal area. This aligns with how structural engineers assess crack severity in practice, where crack length and continuity are more indicative of structural risk than area alone.

The Wall Health Index consolidates all of this into a single intuitive number, making the output directly usable by non-specialist building managers for maintenance planning and prioritization.

9.4 Limitations

This project has several limitations worth acknowledging:

1. The model produces a single defect label per image. It cannot handle cases where multiple defect types appear simultaneously in the same photograph.
2. The 16 classical features are computed over the entire image. For localized defects in one corner of a large wall, this global computation may dilute the signal.
3. The XGBoost comparison was conducted on the validation set rather than the held-

out test set, due to timing constraints. A rigorous comparison would use a shared test set.

4. The dataset comprises photographs collected under relatively controlled conditions. Generalization to images taken in varied lighting, at different camera angles, or under adverse weather has not been validated.

9.5 Future Work

Several directions could extend this work:

1. **Instance segmentation:** Using architectures like Mask R-CNN to detect and localize multiple defects simultaneously in a single image.
2. **Domain adaptation:** Expanding the dataset with real-world images from diverse conditions and applying domain adaptation techniques to bridge the gap between controlled and in-the-wild data.
3. **Mobile deployment:** Packaging the model as a smartphone application using MobileNetV2 quantization, allowing inspectors to submit photographs from the field for immediate classification.
4. **Multi-label classification:** Extending the output layer to support simultaneous prediction of multiple defect types per image.

10 Conclusion

This project presented an automated wall defect classification and severity assessment system covering five common building surface defects: damp, mold, crack, spalling, and peeling. The system combines a fine-tuned MobileNetV2 deep learning model with 16 domain-specific hand-crafted features in a dual-branch fusion architecture. A two-phase training strategy protects the pre-trained backbone during early training, while CLAHE and bilateral filtering preprocessing enhances defect visibility without distorting color.

The hybrid model achieves **89% accuracy** and a **macro F1-score of 0.89** on a balanced 298-image test set, a 10-point improvement over an XGBoost baseline that used classical features alone. The most significant gain was on the challenging spalling–peeling pair, where the CNN's spatial reasoning resolved the ambiguity that classical features could not. Class-specific severity scores and a Wall Health Index (0–100) make the system's output directly actionable for maintenance planning. Grad-CAM visualizations provide transparency, allowing human inspectors to verify predictions and build trust in the system.

Overall, this project demonstrates that combining deep learning with domain-specific

classical features produces a more accurate, interpretable, and practically useful system than either approach alone, with direct potential for application in automated building inspection workflows.

Acknowledgements. We thank the Department of Computer Science and Engineering (Data Science) at MPSTME for providing computational resources and laboratory facilities for this project. We are grateful to our project guide and faculty members for their guidance and support throughout the academic year.

References

[1] W. J. Fisk, Q. Lei-Gomez, and M. J. Mendell, "Meta-analyses of the associations of respiratory health effects with dampness and mold in homes," *Indoor Air*, vol. 17, no. 4, pp. 284–296, 2007.

[2] B. Marteinsson, "Service life estimation in the design of buildings: A development of the factor method," *Building Research & Information*, vol. 33, no. 6, pp. 530–544, 2005.

[3] J. Kim and J. C. P. Cheng, "Automated detection of building damage and defects using deep learning," in *Proc. ISARC*, 2018, pp. 1–8.

[4] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[5] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.

[6] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, 1973.

[7] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, 2002.

[8] A. K. Jain and F. Farrokhnia, "Unsupervised texture segmentation using Gabor filters," *Pattern Recognit.*, vol. 24, no. 12, pp. 1167–1186, 1991.

[9] Y.-J. Cha, W. Choi, and O. Büyüköztürk, "Deep learning-based crack damage detection using convolutional neural networks," *Comput.-Aided Civ. Infrastruct. Eng.*, vol. 32, no. 5, pp. 361–378, 2017.

[10] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010.

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.

[12] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, 2017, pp. 618–626.

[13] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*. Academic Press, 1994, pp. 474–485.

[14] Y. Shi, L. Cui, Z. Qi, F. Meng, and Z. Chen, "Automatic road crack detection using random structured forests," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 12, pp. 3434–3445, 2016.

[15] C. V. Dung and L. D. Anh, "Autonomous concrete crack detection using deep fully convolutional neural network," *Autom. Constr.*, vol. 99, pp. 52–58, 2019.

[16] Z. Liu, Y. Cao, Y. Wang, and W. Wang, "Computer vision-based concrete crack detection using U-net fully convolutional networks," *Autom. Constr.*, vol. 104, pp. 129–139, 2019.

[17] J. Donahue *et al.*, "DeCAF: A deep convolutional activation feature for generic visual recognition," in *Proc. ICML*, 2014, pp. 647–655.

[18] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM KDD*, 2016, pp. 785–794.

[19] A. Mohan and S. Poobal, "Crack detection using image processing: A critical review and analysis," *Alexandria Eng. J.*, vol. 57, no. 2, pp. 787–798, 2018.

[20] T.-C. Lee, R. L. Kashyap, and C.-N. Chu, "Building skeleton models via 3-D medial surface/axis thinning algorithms," *CVGIP: Graph. Models Image Process.*, vol. 56, no. 6, pp. 462–478, 1994.